

CargoTrack

Rebuilding the Database by Hand

A step-by-step replica of the 19-table PostgreSQL schema, with every query, every check, and the reasoning behind each design decision.

PostgreSQL 16 · DBeaver · practice database cargotrack_practice

How to use this guide

The real CargoTrack database is generated by Prisma from `backend/prisma/schema.prisma`. This guide rebuilds the exact same structure in a separate practice database, one group of tables at a time, so that every column, type, index and foreign key is written and understood deliberately.

Each lesson has the same shape: what the tables represent, the SQL to paste, **What to notice** (the design reasoning), and a **check query** that proves the step worked.

Build order

A column can only use a type that exists, and a foreign key can only point to a table that exists. The lessons therefore follow dependency order:

#	Lesson	Creates	Running total
1	Enums	17 enum types	0 tables
2	Parties	clients, users, client_documents, suppliers, freight_providers, customs_agents	6 tables
3	Orders	orders, order_items, production_orders, qc_inspections	10 tables
4	Warehousing	warehouses, stock_records	12 tables
5	Shipping	containers, container_allocations, transit_legs	15 tables
6	Documents, money & audit	documents, payments, status_history, notifications	19 tables
7	Final verification	Totals + automatic comparison with the real database	
8	Clean-up	Drop the practice database	

Setup: DBeaver and the practice database

Connection settings (DBeaver → New Connection → PostgreSQL):

Field	Value	Why
Host	127.0.0.1	Not localhost: on this machine a WSL relay also listens on localhost's IPv6 address and can intercept the connection.
Port	5432	Published by the postgres service in docker-compose.yml
Username	cargotrack	Not DBeaver's default postgres (the cause of "password authentication failed")
Password	POSTGRES_PASSWORD in .env	The .env file in the project root
Database	cargotrack	The real database, used only to create the practice one

Step 1. On the main connection, open a SQL editor (Ctrl + J) and run:

```
CREATE DATABASE cargotrack_practice;
```

Step 2. Copy/paste the connection, edit the copy: Database = `cargotrack_practice`, name it **CargoTrack PRACTICE** and give it a colour. All lessons run on this connection, so the real database is never touched.

Step 3. Confirm you are in the right place:

```
SELECT current_database(); -- must return cargotrack_practice
```

Running scripts in DBeaver: Ctrl + Enter runs only the statement under the cursor. **Alt + X** runs the whole script. Every lesson below should be run with Alt + X.

Lesson 1: Enums

What is an enum?

An **enum** is a custom data type that only allows values from a fixed list. An order status can only be one of 12 values; if anyone tries to save 'ALMOST_DONE', **PostgreSQL itself refuses it**. This is the project rule "status columns are never free strings": bad data is stopped at the database, not only in the code.

The syntax

```
CREATE TYPE "UserRole" AS ENUM ('OFFICE_MANAGER', 'CLIENT');
```

- **The name is in double quotes.** Without quotes PostgreSQL lowercases names (userrole). Prisma keeps the capitals, so an exact replica must quote them.
- **Values are in single quotes and their order matters.** PostgreSQL remembers the order, so ORDER BY status sorts by list position, not alphabetically.

The 17 enums, grouped by purpose

Group	Enum	Values
Who uses the system	UserRole	OFFICE_MANAGER, CLIENT
Lifecycles	OrderStatus	ORDER_PLACED, ORDER_CONFIRMED, GOODS_RECEIVED, SHIPMENT_BOOKING, ROUTE_DECISION, IN_TRANSIT, DELIVERED, CLOSED_OUT, CANCELLED, FACTORY_CANNOT_FULFIL, QC_REJECTED, DOCUMENTS_WITHHELD
	ContainerStatus	OPEN_FOR_ALLOCATION, FULLY_ALLOCATED, DEPARTED, ARRIVED, CLOSED
	ProductionOrderStatus	PENDING, IN_PRODUCTION, READY, RECEIVED, CANCELLED
	StockStatus	IN_STOCK, ON_HOLD, RELEASED
	QcOutcome	PASSED, REJECTED, PARTIAL
Shipping choices	RouteType	DIRECT, TRANSIT
	PricingBasis	CBM, WEIGHT, FLAT
Documents	DocumentType	BILL_OF_LADING, PACKING_LIST, COMMERCIAL_INVOICE, CERTIFICATE_OF_ORIGIN, CUSTOMS_DECLARATION, INSURANCE_CERTIFICATE, QC_REPORT, OTHER
	ClientDocumentType	PASSPORT_SCAN, TRADE_LICENSE, TAX_CARD, SIGNATURE_SPECIMEN, OTHER
Money	PaymentType	DEPOSIT, BALANCE, FREIGHT, CUSTOMS, STORAGE, REFUND
	PaymentDirection	INBOUND, OUTBOUND
	CounterpartyType	CLIENT, SUPPLIER, FREIGHT_PROVIDER, CUSTOMS_AGENT, WAREHOUSE
Audit & notifications	EntityType	ORDER, CONTAINER, PRODUCTION_ORDER, STOCK_RECORD, DOCUMENT, PAYMENT
	NotificationChannel	EMAIL, SMS, WHATSAPP, IN_APP
	NotificationStatus	PENDING, SENT, FAILED, RETRYING, DEAD_LETTER
	RecipientType	CLIENT, USER, SUPPLIER, FREIGHT_PROVIDER, CUSTOMS_AGENT

The SQL

SQL: RUN WITH ALT + X

```
CREATE TYPE "UserRole" AS ENUM ('OFFICE_MANAGER', 'CLIENT');

CREATE TYPE "OrderStatus" AS ENUM (
    'ORDER_PLACED', 'ORDER_CONFIRMED', 'GOODS_RECEIVED', 'SHIPMENT_BOOKING',
    'ROUTE_DECISION', 'IN_TRANSIT', 'DELIVERED', 'CLOSED_OUT',
    'CANCELLED', 'FACTORY_CANNOT_FULFIL', 'QC_REJECTED', 'DOCUMENTS_WITHHELD'
);

CREATE TYPE "ContainerStatus" AS ENUM (
    'OPEN_FOR_ALLOCATION', 'FULLY_ALLOCATED', 'DEPARTED', 'ARRIVED', 'CLOSED'
);

CREATE TYPE "ProductionOrderStatus" AS ENUM (
    'PENDING', 'IN_PRODUCTION', 'READY', 'RECEIVED', 'CANCELLED'
);

CREATE TYPE "QcOutcome" AS ENUM ('PASSED', 'REJECTED', 'PARTIAL');

CREATE TYPE "StockStatus" AS ENUM ('IN_STOCK', 'ON_HOLD', 'RELEASED');

CREATE TYPE "RouteType" AS ENUM ('DIRECT', 'TRANSIT');

CREATE TYPE "PricingBasis" AS ENUM ('CBM', 'WEIGHT', 'FLAT');

CREATE TYPE "DocumentType" AS ENUM (
    'BILL_OF_LADING', 'PACKING_LIST', 'COMMERCIAL_INVOICE',
    'CERTIFICATE_OF_ORIGIN', 'CUSTOMS_DECLARATION', 'INSURANCE_CERTIFICATE',
    'QC_REPORT', 'OTHER'
);

CREATE TYPE "ClientDocumentType" AS ENUM (
    'PASSPORT_SCAN', 'TRADE_LICENSE', 'TAX_CARD', 'SIGNATURE_SPECIMEN', 'OTHER'
);

CREATE TYPE "PaymentType" AS ENUM (
    'DEPOSIT', 'BALANCE', 'FREIGHT', 'CUSTOMS', 'STORAGE', 'REFUND'
);

CREATE TYPE "PaymentDirection" AS ENUM ('INBOUND', 'OUTBOUND');

CREATE TYPE "CounterpartyType" AS ENUM (
    'CLIENT', 'SUPPLIER', 'FREIGHT_PROVIDER', 'CUSTOMS_AGENT', 'WAREHOUSE'
);

CREATE TYPE "EntityType" AS ENUM (
    'ORDER', 'CONTAINER', 'PRODUCTION_ORDER', 'STOCK_RECORD', 'DOCUMENT', 'PAYMENT'
);

CREATE TYPE "NotificationChannel" AS ENUM ('EMAIL', 'SMS', 'WHATSAPP', 'IN_APP');

CREATE TYPE "NotificationStatus" AS ENUM (
    'PENDING', 'SENT', 'FAILED', 'RETRYING', 'DEAD_LETTER'
);

CREATE TYPE "RecipientType" AS ENUM (
    'CLIENT', 'USER', 'SUPPLIER', 'FREIGHT_PROVIDER', 'CUSTOMS_AGENT'
);
```

Why create the enums first?

1. In PostgreSQL an enum is a separate object; it cannot be typed inside a column. MySQL allows status ENUM('A', 'B') inline. PostgreSQL does not: an enum is a **named type**, like a custom TEXT or INTEGER, and a column can only use a type that already exists, just as a foreign key can only point to an existing table.

2. Defined once, reused everywhere. EntityType is used by both status_history and notifications. Adding a value later is one statement, and every column using the type gets it:

```
ALTER TYPE "EntityType" ADD VALUE 'WAREHOUSE';
```

The **inline alternative** is a check constraint, `status TEXT CHECK (status IN ('PENDING', 'SENT', ...))`. It works, but the list is copied into every column that needs it, and copies drift apart.

Check

CHECK QUERY

```
SELECT t.typname AS enum_name,  
       string_agg(e.enumlabel, ', ' ORDER BY e.enumsortorder) AS enum_values  
FROM pg_type t  
JOIN pg_enum e ON e.enumtypid = t.oid  
GROUP BY t.typname  
ORDER BY t.typname;
```

Expected: **17 rows**. To redo one: `DROP TYPE "Name";`

How the check query works

PostgreSQL stores information *about* the database in its own tables, the **system catalog** (names starting with `pg_`).

Catalog table	What it holds	Columns used
<code>pg_type</code>	Every data type (built-in and yours)	<code>oid</code> = internal ID, <code>typname</code> = name
<code>pg_enum</code>	One row per enum value	<code>enumtypid</code> = owning type, <code>enumlabel</code> = value, <code>enumsortorder</code> = position

```
FROM pg_type t                                -- start from all types, nickname "t"  
JOIN pg_enum e ON e.enumtypid = t.oid         -- attach each enum value to its type  
                                              -- (non-enum types have no match and disappear)  
GROUP BY t.typname                           -- squash all values of one type into one row  
SELECT t.typname AS enum_name,  
       string_agg(e.enumlabel, ', '          -- glue the squashed values into one text  
                  ORDER BY e.enumsortorder) AS enum_values -- in definition order  
ORDER BY t.typname;                          -- sort result by name
```

Without `GROUP BY + string_agg` you would get one row per value (76 rows). `string_agg` is an **aggregate function**: it combines many rows into one value, like `COUNT` or `SUM`.

Lesson 2: Parties

The people and companies the system deals with.

Table	Who	Notes
clients	Companies buying through the office	The anchor of data scoping
users	People who can log in	Office manager, or a client's employee linked to a client
client_documents	Passport scans, trade licenses...	Sensitive; kept apart so they can expire
suppliers	Factories that make the goods	
freight_providers	Shipping lines	
customs_agents	Customs brokers	

SQL: RUN WITH ALT + X

```
-- ===== TABLES =====

CREATE TABLE "clients" (
  "id" UUID NOT NULL,
  "company_name" TEXT NOT NULL,
  "contact_name" TEXT NOT NULL,
  "email" TEXT NOT NULL,
  "phone" TEXT,
  "country" TEXT NOT NULL,
  "address" TEXT,
  "is_consolidator" BOOLEAN NOT NULL DEFAULT false,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "clients_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "users" (
  "id" UUID NOT NULL,
  "name" TEXT NOT NULL,
  "email" TEXT NOT NULL,
  "password_hash" TEXT NOT NULL,
  "role" "UserRole" NOT NULL,
  "client_id" UUID,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "users_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "client_documents" (
  "id" UUID NOT NULL,
  "client_id" UUID NOT NULL,
  "doc_type" "ClientDocumentType" NOT NULL,
  "file_ref" TEXT NOT NULL,
  "uploaded_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "retention_expires_at" TIMESTAMP(3),

  CONSTRAINT "client_documents_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "suppliers" (
  "id" UUID NOT NULL,
  "name" TEXT NOT NULL,
  "country" TEXT NOT NULL,
  "contact_phone" TEXT,
  "notes" TEXT,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "suppliers_pkey" PRIMARY KEY ("id")
);
```

```

CREATE TABLE "freight_providers" (
  "id" UUID NOT NULL,
  "name" TEXT NOT NULL,
  "country" TEXT NOT NULL,
  "contact_phone" TEXT,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "freight_providers_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "customs_agents" (
  "id" UUID NOT NULL,
  "name" TEXT NOT NULL,
  "country" TEXT NOT NULL,
  "phone" TEXT,
  "notes" TEXT,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "customs_agents_pkey" PRIMARY KEY ("id")
);

-- ===== INDEXES =====

CREATE UNIQUE INDEX "users_email_key" ON "users"("email");
CREATE INDEX "users_client_id_idx" ON "users"("client_id");
CREATE INDEX "users_role_idx" ON "users"("role");
CREATE INDEX "clients_company_name_idx" ON "clients"("company_name");
CREATE INDEX "client_documents_client_id_idx" ON "client_documents"("client_id");
CREATE INDEX "client_documents_retention_expires_at_idx"
  ON "client_documents"("retention_expires_at");
CREATE INDEX "suppliers_name_idx" ON "suppliers"("name");
CREATE INDEX "freight_providers_name_idx" ON "freight_providers"("name");
CREATE INDEX "customs_agents_name_idx" ON "customs_agents"("name");

-- ===== FOREIGN KEYS =====

ALTER TABLE "users" ADD CONSTRAINT "users_client_id_fkey"
  FOREIGN KEY ("client_id") REFERENCES "clients"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE "client_documents" ADD CONSTRAINT "client_documents_client_id_fkey"
  FOREIGN KEY ("client_id") REFERENCES "clients"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

```

What to notice

- 1. id UUID has no default. Who generates it?** The **application**. In Prisma it is `@default(uuid())`, filled in before the INSERT is sent. The database only requires that an id is given (NOT NULL) and unique (PRIMARY KEY).
- 2. updated_at has no default either.** Prisma's `@updatedAt` sets it on every update. `created_at` does have a database default (CURRENT_TIMESTAMP) because it is set once.
- 3. TIMESTAMP(3):** the (3) means millisecond precision, e.g. 2026-09-25 14:03:07.123.
- 4. NOT NULL vs nothing:** NOT NULL = required, nothing = optional. Note which are optional: phone, address, notes, and `users.client_id`, because an office manager belongs to no client.
- 5. Indexes are the database's table of contents.**
 - A normal INDEX makes searching/filtering by that column fast ("all users of client X", "suppliers named ...").
 - A UNIQUE INDEX also **forbids duplicates**: `users_email_key` is why two users cannot share an email.
 - `client_documents_retention_expires_at_idx` exists for the cleanup job that will ask "which documents have expired?"
- 6. Foreign keys come last, via ALTER TABLE.** Prisma adds all foreign keys after every table exists, so the order of the CREATE TABLE statements never matters. A foreign key says "this column must point to a real row in that table".

7. ON DELETE: what happens to children when the parent is deleted.

Relation	Rule	Meaning
users → clients	SET NULL	Delete a client and their users remain, just unlinked (client_id becomes empty).
client_documents → clients	CASCADE	Delete a client and their passport scans are deleted too: sensitive data must not outlive its owner.

ON UPDATE CASCADE: if a parent's id ever changed, the children would follow. With UUIDs this practically never happens.

Check

CHECK QUERY

```
SELECT table_name, COUNT(*) AS columns
FROM information_schema.columns
WHERE table_schema = 'public'
GROUP BY table_name
ORDER BY table_name;
```

Expected: 6 tables: client_documents (6 columns), clients (10), customs_agents (7), freight_providers (6), suppliers (7), users (8).

Also try: right-click the PRACTICE connection → Refresh → Schemas → public → Tables → any table → **ER Diagram** tab. users and client_documents appear connected to clients.

Lesson 3: Orders

The commercial core. The four tables form a chain (—< means one-to-many):

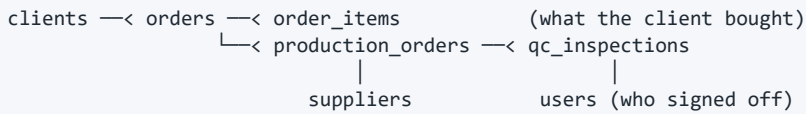


Table	What it represents
orders	A client's purchase: price, currency, deposit and current status
order_items	The lines: what, how many, size / weight / price per unit
production_orders	A batch placed with a factory to make (part of) the order
qc_inspections	Quality checks on a batch, and the client's sign-off

SQL: RUN WITH ALT + X

```
-- ===== TABLES =====

CREATE TABLE "orders" (
  "id" UUID NOT NULL,
  "client_id" UUID NOT NULL,
  "status" "OrderStatus" NOT NULL DEFAULT 'ORDER_PLACED',
  "agreed_price" DECIMAL(14,2) NOT NULL,
  "currency" CHAR(3) NOT NULL,
  "deposit_percentage" DECIMAL(5,2) NOT NULL DEFAULT 20,
  "total_cbm" DECIMAL(10,3),
  "total_weight_kg" DECIMAL(12,3),
  "required_by" DATE,
  "placed_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "closed_at" TIMESTAMP(3),
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "orders_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "order_items" (
  "id" UUID NOT NULL,
  "order_id" UUID NOT NULL,
  "description" TEXT NOT NULL,
  "category" TEXT,
  "quantity" INTEGER NOT NULL,
  "unit_cbm" DECIMAL(10,4) NOT NULL,
  "unit_weight_kg" DECIMAL(12,3) NOT NULL,
  "unit_price" DECIMAL(14,2) NOT NULL,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "order_items_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "production_orders" (
  "id" UUID NOT NULL,
  "order_id" UUID NOT NULL,
  "supplier_id" UUID NOT NULL,
  "agreed_cost" DECIMAL(14,2) NOT NULL,
  "currency" CHAR(3) NOT NULL,
  "expected_ready_date" DATE,
  "actual_received_date" DATE,
  "status" "ProductionOrderStatus" NOT NULL DEFAULT 'PENDING',
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "production_orders_pkey" PRIMARY KEY ("id")
);
```

```

CREATE TABLE "qc_inspections" (
  "id" UUID NOT NULL,
  "production_order_id" UUID NOT NULL,
  "inspected_at" TIMESTAMP(3) NOT NULL,
  "outcome" "QcOutcome" NOT NULL,
  "rejection_notes" TEXT,
  "client_signed_off_at" TIMESTAMP(3),
  "signed_off_by" UUID,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "qc_inspections_pkey" PRIMARY KEY ("id")
);

-- ===== INDEXES =====

CREATE INDEX "orders_client_id_idx" ON "orders"("client_id");
CREATE INDEX "orders_status_idx" ON "orders"("status");
CREATE INDEX "orders_placed_at_idx" ON "orders"("placed_at");
CREATE INDEX "order_items_order_id_idx" ON "order_items"("order_id");
CREATE INDEX "production_orders_order_id_idx" ON "production_orders"("order_id");
CREATE INDEX "production_orders_supplier_id_idx" ON "production_orders"("supplier_id");
CREATE INDEX "production_orders_status_idx" ON "production_orders"("status");
CREATE INDEX "qc_inspections_production_order_id_idx"
  ON "qc_inspections"("production_order_id");

-- ===== FOREIGN KEYS =====

ALTER TABLE "orders" ADD CONSTRAINT "orders_client_id_fkey"
  FOREIGN KEY ("client_id") REFERENCES "clients"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE "order_items" ADD CONSTRAINT "order_items_order_id_fkey"
  FOREIGN KEY ("order_id") REFERENCES "orders"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "production_orders" ADD CONSTRAINT "production_orders_order_id_fkey"
  FOREIGN KEY ("order_id") REFERENCES "orders"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "production_orders" ADD CONSTRAINT "production_orders_supplier_id_fkey"
  FOREIGN KEY ("supplier_id") REFERENCES "suppliers"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE "qc_inspections" ADD CONSTRAINT "qc_inspections_production_order_id_fkey"
  FOREIGN KEY ("production_order_id") REFERENCES "production_orders"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "qc_inspections" ADD CONSTRAINT "qc_inspections_signed_off_by_fkey"
  FOREIGN KEY ("signed_off_by") REFERENCES "users"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

```

What to notice

1. Money and measurements use DECIMAL, never floats. DECIMAL(14, 2) = up to 14 digits in total, 2 after the point (max 999,999,999,999.99). Decimals are **exact**; a float stores 0.1 + 0.2 as 0.30000000000000004, unacceptable for money, and the reason the Phase 3 container capacity guard also uses Decimal.

Column	Type	Why
agreed_price, unit_price, agreed_cost	DECIMAL(14,2)	Money has cents
unit_cbm	DECIMAL(10,4)	One chair can be 0.0850 m³: 4 decimals needed
total_cbm	DECIMAL(10,3)	Totals need less precision
deposit_percentage	DECIMAL(5,2)	e.g. 22.50

2. Every money column has its own currency. CHAR(3) = exactly 3 characters, an ISO code (USD, CNY). The client pays in USD while the factory is paid in CNY, so each amount must say which currency it is.

3. **DATE vs TIMESTAMP.** `required_by` and `expected_ready_date` are calendar days (no time needed); `placed_at` and `inspected_at` are moments.

4. **Enum columns with a default.** `status "OrderStatus" DEFAULT 'ORDER_PLACED'`: every new order starts at the first state automatically; production orders start at 'PENDING'.

5. **`total_cbm` and `total_weight_kg` are optional and calculated.** Nobody types them: the backend recomputes them from the items (`quantity × unit_cbm`) whenever an item changes. They stay empty until the order has items.

6. **A new delete rule:** `RESTRICT`.

Relation	Rule	Meaning
<code>orders → clients</code>	<code>RESTRICT</code>	Cannot delete a client who has orders: business history must not vanish.
<code>production_orders → suppliers</code>	<code>RESTRICT</code>	Cannot delete a factory that has produced for you.
<code>order_items / production_orders → orders</code>	<code>CASCADE</code>	Delete an order and its lines and batches go too (the API only allows it while <code>ORDER_PLACED</code>).
<code>qc_inspections → production_orders</code>	<code>CASCADE</code>	Inspections belong to their batch.
<code>qc_inspections.signed_off_by → users</code>	<code>SET NULL</code>	If the user is removed, the sign-off stays but loses its "who".

The backend turns a `RESTRICT` failure into **HTTP 409 Conflict** (the Prisma exception filter).

7. **What is not in the database.** Some key rules live in the backend services, not here:

- the deposit must be between 15% and 25%;
- status can only move along the state diagram (the state machines);
- a shipment cannot be booked until a passed inspection has `client_signed_off_at` set.

The database guarantees the **shape** of the data. The services guarantee the **business rules**.

Check

CHECK QUERY

```
SELECT tc.table_name, tc.constraint_name,
       ccu.table_name AS points_to, rc.delete_rule
FROM information_schema.table_constraints tc
JOIN information_schema.referential_constraints rc
  ON rc.constraint_name = tc.constraint_name
JOIN information_schema.constraint_column_usage ccu
  ON ccu.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY'
ORDER BY tc.table_name;
```

Lists every foreign key with its delete rule. Expected at this point: **8 rows** (2 from parties + 6 new).

Lesson 4: Warehousing

warehouses —< stock_records >— order_items

Table	What it represents
warehouses	A physical storage place, e.g. a bonded warehouse in Alexandria or one in Guangzhou
stock_records	"X units of this order line are in this warehouse", plus held / released status

SQL: RUN WITH ALT + X

```
-- ===== TABLES =====

CREATE TABLE "warehouses" (
  "id" UUID NOT NULL,
  "name" TEXT NOT NULL,
  "country" TEXT NOT NULL,
  "address" TEXT,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "warehouses_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "stock_records" (
  "id" UUID NOT NULL,
  "warehouse_id" UUID NOT NULL,
  "order_item_id" UUID NOT NULL,
  "quantity" INTEGER NOT NULL,
  "status" "StockStatus" NOT NULL DEFAULT 'IN_STOCK',
  "hold_reason" TEXT,
  "received_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "released_at" TIMESTAMP(3),
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "stock_records_pkey" PRIMARY KEY ("id")
);

-- ===== INDEXES =====

CREATE INDEX "stock_records_warehouse_id_idx" ON "stock_records"("warehouse_id");
CREATE INDEX "stock_records_order_item_id_idx" ON "stock_records"("order_item_id");
CREATE INDEX "stock_records_status_idx" ON "stock_records"("status");

-- ===== FOREIGN KEYS =====

ALTER TABLE "stock_records" ADD CONSTRAINT "stock_records_warehouse_id_fkey"
  FOREIGN KEY ("warehouse_id") REFERENCES "warehouses"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;

ALTER TABLE "stock_records" ADD CONSTRAINT "stock_records_order_item_id_fkey"
  FOREIGN KEY ("order_item_id") REFERENCES "order_items"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
```

What to notice

- 1. Stock points at order_items, not at orders.** A warehouse holds physical goods, and goods are order *lines* ("100 of the 120 rattan chairs"), so you know exactly **what** is on the shelf and **how many**.
- 2. A table that connects two others but carries its own data.** The same item can have several records: 100 units in March and 20 in April, or some in Guangzhou and some in Alexandria.
- 3. The stock lifecycle lives in status + hold_reason + two timestamps.**

Column	Role
status	IN_STOCK → ON_HOLD ⇄ IN_STOCK → RELEASED (the Phase 3 state machine)

Column	Role
hold_reason	Free text on purpose: "client waiting for Ramadan demand", "customs query"; a fixed list could not cover real reasons
received_at → released_at	Together they measure how long goods sat there: the roadmap's "seasonal hold tracking". released_at is optional because goods on the shelf are not released yet.

4. Delete rules.

Relation	Rule	Meaning
stock_records → warehouses	RESTRICT	Cannot delete a warehouse with stock history (the Gate 3 test expects 409).
stock_records → order_items	CASCADE	If an order line is removed, its stock records go with it.

5. An observation. warehouses has no index on name, unlike the parties. There will only ever be a handful of warehouses; PostgreSQL scans them instantly, so an index would add nothing.

6. Not in the database, again. "No more units in warehouses than were ordered on that line" is enforced in stock-records.service.ts. The database would accept 500 chairs on a line that ordered 120.

Check

CHECK QUERY

```
SELECT COUNT(*) AS tables
FROM information_schema.tables
WHERE table_schema = 'public';
```

Expected: **12** (6 parties + 4 orders + 2 warehousing).

Lesson 5: Shipping

The container side of the business: one box, many orders, an optional route through transit ports.

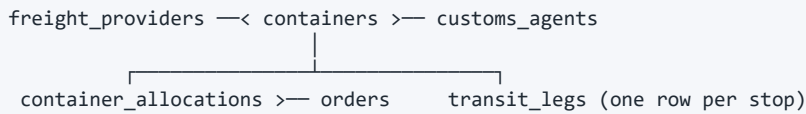


Table	What it represents
containers	One physical box booked with a freight provider: capacity, route, cost, lifecycle status
container_allocations	"This order occupies X CBM and Y kg of this container": consolidated shipping
transit_legs	One row per intermediate port when the route is TRANSIT (e.g. Jebel Ali)

SQL: RUN WITH ALT + X

```
-- ===== TABLES =====

CREATE TABLE "containers" (
  "id" UUID NOT NULL,
  "freight_provider_id" UUID,
  "customs_agent_id" UUID,
  "container_ref" TEXT NOT NULL,
  "container_type" TEXT NOT NULL,
  "capacity_cbm" DECIMAL(10,3) NOT NULL,
  "capacity_weight_kg" DECIMAL(12,3) NOT NULL,
  "pricing_basis" "PricingBasis" NOT NULL DEFAULT 'CBM',
  "booking_cost" DECIMAL(14,2),
  "currency" CHAR(3),
  "origin_port" TEXT NOT NULL,
  "destination_port" TEXT NOT NULL,
  "route_type" "RouteType" NOT NULL DEFAULT 'DIRECT',
  "insurance_ref" TEXT,
  "status" "ContainersStatus" NOT NULL DEFAULT 'OPEN_FOR_ALLOCATION',
  "booked_at" TIMESTAMP(3),
  "departed_at" TIMESTAMP(3),
  "arrived_at" TIMESTAMP(3),
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "containers_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "container_allocations" (
  "id" UUID NOT NULL,
  "container_id" UUID NOT NULL,
  "order_id" UUID NOT NULL,
  "allocated_cbm" DECIMAL(10,3) NOT NULL,
  "allocated_weight_kg" DECIMAL(12,3) NOT NULL,
  "allocated_cost" DECIMAL(14,2),
  "currency" CHAR(3),
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "container_allocations_pkey" PRIMARY KEY ("id")
);
```

```

CREATE TABLE "transit_legs" (
  "id" UUID NOT NULL,
  "container_id" UUID NOT NULL,
  "sequence" INTEGER NOT NULL,
  "port" TEXT NOT NULL,
  "arrived_at" TIMESTAMP(3),
  "departed_at" TIMESTAMP(3),
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "transit_legs_pkey" PRIMARY KEY ("id")
);

-- ===== INDEXES =====

CREATE UNIQUE INDEX "containers_container_ref_key" ON "containers"("container_ref");
CREATE INDEX "containers_status_idx" ON "containers"("status");
CREATE INDEX "containers_freight_provider_id_idx" ON "containers"("freight_provider_id");
CREATE INDEX "container_allocations_order_id_idx" ON "container_allocations"("order_id");
CREATE UNIQUE INDEX "container_allocations_container_id_order_id_key"
  ON "container_allocations"("container_id", "order_id");
CREATE UNIQUE INDEX "transit_legs_container_id_sequence_key"
  ON "transit_legs"("container_id", "sequence");

-- ===== FOREIGN KEYS =====

ALTER TABLE "containers" ADD CONSTRAINT "containers_freight_provider_id_fkey"
  FOREIGN KEY ("freight_provider_id") REFERENCES "freight_providers"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE "containers" ADD CONSTRAINT "containers_customs_agent_id_fkey"
  FOREIGN KEY ("customs_agent_id") REFERENCES "customs_agents"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE "container_allocations" ADD CONSTRAINT "container_allocations_container_id_fkey"
  FOREIGN KEY ("container_id") REFERENCES "containers"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "container_allocations" ADD CONSTRAINT "container_allocations_order_id_fkey"
  FOREIGN KEY ("order_id") REFERENCES "orders"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "transit_legs" ADD CONSTRAINT "transit_legs_container_id_fkey"
  FOREIGN KEY ("container_id") REFERENCES "containers"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

```

What to notice

1. container_allocations is the many-to-many junction. One container carries many orders (several clients share a box), and in principle one order can be split across several containers. Such a relationship cannot be stored as a column on either side, so it gets its own table, and that table also carries the **size of each share** (allocated_cbm, allocated_weight_kg) and optionally its **cost**.

2. New: a UNIQUE index across two columns.

Index	What it forbids
container_allocations (container_id, order_id)	The same order twice in the same container. Different containers for one order are still allowed.
transit_legs (container_id, sequence)	Two "stop #1"s on the same container. Every container can still have its own stop #1.
containers (container_ref)	Two containers with the same box number (e.g. MSKU-4471820).

The **combination** must be unique, not each column alone. The backend turns a violation into **409 Conflict**: the Gate 3 test "refuses to put the same order in the same box twice" relies on exactly this index.

3. Capacity is two limits, not one. capacity_cbm and capacity_weight_kg: a box of steel runs out of payload before volume, a box of rattan the opposite. The rule "the sum of allocations may not exceed either limit" spans many rows, so no column constraint can express it. It is the **capacity guard** in the backend, which also locks the

container row (SELECT ... FOR UPDATE) so two simultaneous allocations cannot both squeeze in.

4. Timestamps record the lifecycle. booked_at, departed_at and arrived_at are all optional: each is stamped by the backend at the moment the container moves into FULLY_ALLOCATED, DEPARTED or ARRIVED. A container still open has none of them.

5. Route decision lives on the container. route_type defaults to DIRECT. A DIRECT container has no transit_legs rows; a TRANSIT one has one per stop, each with its own arrived_at / departed_at.

6. Optional money, still paired with a currency. booking_cost and allocated_cost are optional (a price may not be known yet), so their currency columns are optional too. The backend requires the currency whenever a cost is given.

7. Delete rules.

Relation	Rule	Meaning
containers → freight_providers / customs_agents	SET NULL	Removing a provider or agent keeps the container, just unassigned.
container_allocations → containers	CASCADE	The allocations belong to the box.
container_allocations → orders	CASCADE	If an order is removed, its place in the box is freed.
transit_legs → containers	CASCADE	Stops belong to the box.

Check

CHECK QUERY

```
SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
  AND indexdef LIKE 'CREATE UNIQUE%'
  AND indexname NOT LIKE '%_pkey'
ORDER BY tablename;
```

Lists every unique index that is not a primary key. Expected at this point: **4 rows**: users_email_key, containers_container_ref_key, and the two two-column indexes above.

Lesson 6: Documents, money & audit

The cross-cutting tables. They attach to orders, containers or to *anything*.

Table	What it represents
documents	Bills of lading, invoices, packing lists... with version history
payments	Every money movement: deposits and balances in, freight and customs out
status_history	Append-only audit trail: one row for every status change of every entity
notifications	Append-only queue of messages to send, with retry and dead-letter tracking

SQL: RUN WITH ALT + X

```
-- ===== TABLES =====

CREATE TABLE "documents" (
  "id" UUID NOT NULL,
  "order_id" UUID,
  "container_id" UUID,
  "doc_type" "DocumentType" NOT NULL,
  "version" INTEGER NOT NULL DEFAULT 1,
  "supersedes_id" UUID,
  "file_ref" TEXT NOT NULL,
  "prepared_by" UUID,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "released_to_client_at" TIMESTAMP(3),
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "documents_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "payments" (
  "id" UUID NOT NULL,
  "order_id" UUID NOT NULL,
  "payment_type" "PaymentType" NOT NULL,
  "direction" "PaymentDirection" NOT NULL,
  "counterparty_type" "CounterpartyType" NOT NULL,
  "counterparty_id" UUID NOT NULL,
  "amount" DECIMAL(14,2) NOT NULL,
  "currency" CHAR(3) NOT NULL,
  "fx_rate" DECIMAL(14,6),
  "paid_at" TIMESTAMP(3),
  "reference" TEXT,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at" TIMESTAMP(3) NOT NULL,

  CONSTRAINT "payments_pkey" PRIMARY KEY ("id")
);

CREATE TABLE "status_history" (
  "id" UUID NOT NULL,
  "entity_type" "EntityType" NOT NULL,
  "entity_id" UUID NOT NULL,
  "from_status" TEXT,
  "to_status" TEXT NOT NULL,
  "changed_by" UUID,
  "reason" TEXT,
  "changed_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT "status_history_pkey" PRIMARY KEY ("id")
);
```

```

CREATE TABLE "notifications" (
  "id" UUID NOT NULL,
  "event_type" TEXT NOT NULL,
  "entity_type" "EntityType" NOT NULL,
  "entity_id" UUID NOT NULL,
  "recipient_type" "RecipientType" NOT NULL,
  "recipient_id" UUID NOT NULL,
  "channel" "NotificationChannel" NOT NULL,
  "client_visible" BOOLEAN NOT NULL DEFAULT true,
  "status" "NotificationStatus" NOT NULL DEFAULT 'PENDING',
  "retry_count" INTEGER NOT NULL DEFAULT 0,
  "scheduled_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "sent_at" TIMESTAMP(3),
  "last_error" TEXT,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT "notifications_pkey" PRIMARY KEY ("id")
);

-- ===== INDEXES =====

CREATE UNIQUE INDEX "documents_supersedes_id_key" ON "documents"("supersedes_id");
CREATE INDEX "documents_order_id_idx" ON "documents"("order_id");
CREATE INDEX "documents_container_id_idx" ON "documents"("container_id");
CREATE INDEX "documents_doc_type_idx" ON "documents"("doc_type");
CREATE INDEX "payments_order_id_idx" ON "payments"("order_id");
CREATE INDEX "payments_payment_type_idx" ON "payments"("payment_type");
CREATE INDEX "payments_paid_at_idx" ON "payments"("paid_at");
CREATE INDEX "status_history_entity_type_entity_id_idx"
  ON "status_history"("entity_type", "entity_id");
CREATE INDEX "status_history_changed_at_idx" ON "status_history"("changed_at");
CREATE INDEX "notifications_status_idx" ON "notifications"("status");
CREATE INDEX "notifications_entity_type_entity_id_idx"
  ON "notifications"("entity_type", "entity_id");
CREATE INDEX "notifications_scheduled_at_idx" ON "notifications"("scheduled_at");

-- ===== FOREIGN KEYS =====

ALTER TABLE "documents" ADD CONSTRAINT "documents_order_id_fkey"
  FOREIGN KEY ("order_id") REFERENCES "orders"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "documents" ADD CONSTRAINT "documents_container_id_fkey"
  FOREIGN KEY ("container_id") REFERENCES "containers"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE "documents" ADD CONSTRAINT "documents_prepared_by_fkey"
  FOREIGN KEY ("prepared_by") REFERENCES "users"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE "documents" ADD CONSTRAINT "documents_supersedes_id_fkey"
  FOREIGN KEY ("supersedes_id") REFERENCES "documents"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

ALTER TABLE "payments" ADD CONSTRAINT "payments_order_id_fkey"
  FOREIGN KEY ("order_id") REFERENCES "orders"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE "status_history" ADD CONSTRAINT "status_history_changed_by_fkey"
  FOREIGN KEY ("changed_by") REFERENCES "users"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;

```

What to notice

documents

1. A table that points at itself: the version chain. `supersedes_id` references `documents(id)`. A corrected invoice is a new row whose `supersedes_id` is the old invoice, so the versions form a chain $v1 \leftarrow v2 \leftarrow v3$ and nothing is overwritten.

2. The UNIQUE index on `supersedes_id` means a document can be superseded by **at most one** newer version: the chain can never fork into two competing "v2"s. (NULL is not counted as a duplicate, so any number of first

versions can have `supersedes_id` empty.)

3. `order_id` and `container_id` are both optional. A document belongs to an order (commercial invoice), to a container (bill of lading), or to both.

4. The strongest business rule in the domain. `released_to_client_at` stays NULL until the balance payment clears, and until then the backend withholds `file_ref` from the client. The column records the fact; enforcing it is Phase 4 code.

payments

5. A "polymorphic" reference: `counterparty_type` + `counterparty_id`, with no foreign key. The other side of a payment can be a client, supplier, freight provider, customs agent or warehouse, five different tables. A foreign key must point at exactly one table, so the type is stored in an enum and the id in a plain UUID. The trade-off: the database cannot check that the id exists; the backend must.

6. `direction` separates money coming in (client deposit, balance) from money going out (factory, freight, customs). `fx_rate` DECIMAL(14,6) keeps 6 decimals because exchange rates need them (e.g. 0.140845 USD per CNY). `paid_at` is optional: a payment can be recorded as due before it clears.

status_history

7. Append-only: no `updated_at` column. Rows are only ever inserted; a correction is a new row. That is what makes it trustworthy as an audit trail.

8. The same polymorphic pattern: `entity_type` + `entity_id` can point at an order, container, production order, stock record, document or payment, so there is no foreign key on `entity_id`. The composite index on (`entity_type`, `entity_id`) makes "the full history of this container" a fast lookup.

9. `from_status` and `to_status` are TEXT, not an enum. They must hold values from several different enums (OrderStatus, ContainerStatus, StockStatus...); no single enum type can. `from_status` is optional: the first row, written when the entity is created, has nothing to come from.

10. `changed_by` → users with SET NULL: removing a user keeps the history but loses the "who".

notifications

11. Append-only too (no `updated_at`), with the same `entity_type` + `entity_id` pattern and a polymorphic `recipient_type` + `recipient_id`.

12. The retry pipeline is in the columns: `status` (PENDING → SENT, or FAILED → RETRYING → DEAD_LETTER), `retry_count`, `last_error`, `scheduled_at` and `sent_at`. The indexes on `status` and `scheduled_at` serve the worker's question "which messages are due?" and the dashboard's "which ones failed?".

13. notifications has no foreign keys at all. Everything it references is polymorphic.

Delete rules

Relation	Rule	Meaning
documents → orders	CASCADE	An order's documents go with it.
documents → containers	SET NULL	Removing a container keeps its documents.
documents.prepared_by → users	SET NULL	Keeps the document, loses the author.
documents.supersedes_id → documents	SET NULL	Deleting an old version does not delete the newer one.
payments → orders	CASCADE	An order's payments go with it.
status_history.changed_by → users	SET NULL	Keeps the history, loses the "who".

Check

CHECK QUERY

```
SELECT tc.table_name, tc.constraint_name,  
       ccu.table_name AS points_to, rc.delete_rule  
FROM information_schema.table_constraints tc  
JOIN information_schema.referential_constraints rc  
  ON rc.constraint_name = tc.constraint_name  
JOIN information_schema.constraint_column_usage ccu  
  ON ccu.constraint_name = tc.constraint_name  
WHERE tc.constraint_type = 'FOREIGN KEY'  
ORDER BY tc.table_name;
```

Expected now: **21 rows**, every foreign key in the schema.

Lesson 7: Final verification

1. The totals

CHECK QUERY

```
SELECT
  (SELECT COUNT(*) FROM information_schema.tables
   WHERE table_schema = 'public') AS tables,          -- 19
  (SELECT COUNT(*) FROM pg_type t JOIN pg_namespace n ON n.oid = t.typnamespace
   WHERE n.nspname = 'public' AND t.typtype = 'e') AS enums,          -- 17
  (SELECT COUNT(*) FROM information_schema.columns
   WHERE table_schema = 'public') AS columns,          -- 185
  (SELECT COUNT(*) FROM information_schema.table_constraints
   WHERE table_schema = 'public'
     AND constraint_type = 'FOREIGN KEY') AS foreign_keys,          -- 21
  (SELECT COUNT(*) FROM pg_indexes
   WHERE schemaname = 'public') AS indexes;          -- 57
```

Object	Expected	Made of
tables	19	6 parties + 4 orders + 2 warehousing + 3 shipping + 4 cross-cutting
enums	17	Lesson 1
columns	185	All columns of all 19 tables
foreign_keys	21	2 + 6 + 2 + 5 + 6
indexes	57	19 primary keys + 5 unique + 33 normal indexes

2. Automatic comparison with the real database

Totals can match while a detail differs. The honest test is to export the **structure** of both databases and compare them line by line:

```
# Run from the project root (C:\Users\moust\CargoTrack), in Git Bash.
# 1. Dump the structure (no data) of both databases.
docker compose exec -T postgres pg_dump -U cargotrack -d cargotrack \
  --schema-only --no-owner --restrict-key=compare \
  --exclude-table=_prisma_migrations > real.sql

docker compose exec -T postgres pg_dump -U cargotrack -d cargotrack_practice \
  --schema-only --no-owner --restrict-key=compare > practice.sql

# 2. Compare them. No output at all means the two schemas are identical.
git diff --no-index real.sql practice.sql

# 3. Remove the two temporary files.
rm real.sql practice.sql
```

--schema-only exports structure without data. --exclude-table=_prisma_migrations leaves out the bookkeeping table Prisma keeps in the real database to remember which migrations ran; it is not part of the domain, so the replica deliberately does not have it. --restrict-key=compare fixes a security key that newer pg_dump versions otherwise generate randomly in every export, which would make the two files differ on that line alone. **No output from git diff = identical schemas.**

Lesson 8: Clean-up

When you are done, disconnect the PRACTICE connection in DBeaver, then on the **main** connection run:

```
-- Run this on the MAIN connection (database: cargotrack),
-- never on the PRACTICE connection itself.
DROP DATABASE cargotrack_practice WITH (FORCE);
```

WITH (FORCE) (PostgreSQL 13+) closes any session still connected to it. Finally delete the PRACTICE connection in DBeaver. The real cargotrack database is untouched throughout.

Summary: the design patterns in one page

Pattern	Where	Why
Enums for every status	all lifecycle columns	The database refuses invalid values
DECIMAL for money & measures	prices, CBM, weight, fx	Exact arithmetic, no float drift
A currency beside every amount	orders, payments, containers...	Multi-currency trade (USD in, CNY out)
App-generated UUIDs + @updatedAt	every table	Prisma fills id and updated_at, not the DB
FKs added last with ALTER TABLE	all relations	Table creation order never matters
RESTRICT on history owners	clients, suppliers, warehouses	Business history cannot be deleted away
CASCADE on owned children	items, allocations, legs	Children cannot outlive their parent
SET NULL on optional links	users, providers, authors	Keep the record, drop the link
Junction table with data	container_allocations, stock_records	Many-to-many plus quantities
Composite UNIQUE	(container, order), (container, sequence)	Uniqueness of a combination
Self-reference + UNIQUE	documents.supersedes_id	Linear version chain, never forked
Polymorphic type + id (no FK)	payments, status_history, notifications	Point at one of many tables
Append-only (no updated_at)	status_history, notifications	Trustworthy audit and delivery log
Rules in code, not schema	state machines, capacity, deposit band	Rules spanning rows or needing context